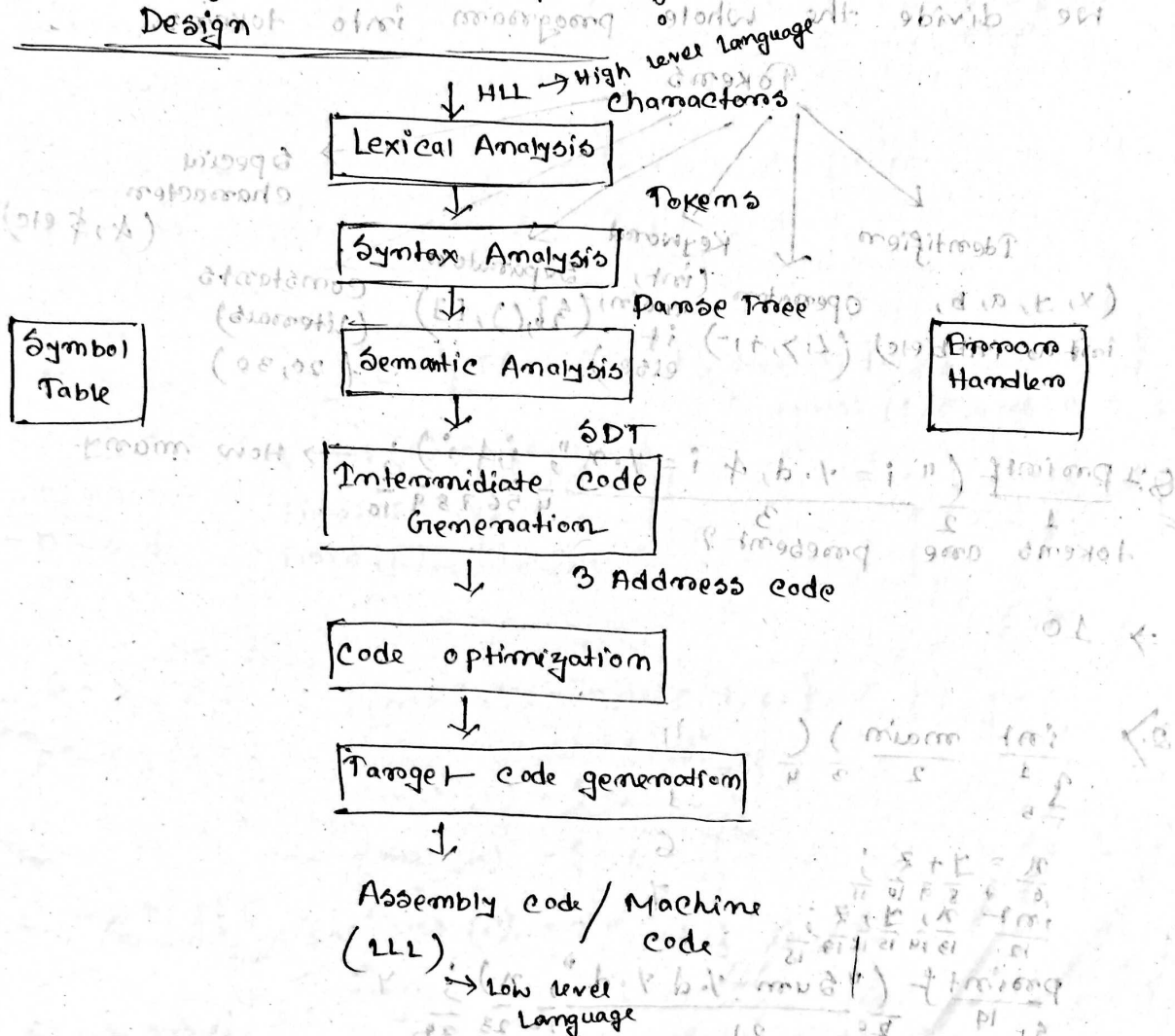


Compiler Design

- (1) Lexical Analysis
- (2) Parser (Syntax Analyzer)
- (3) Semantic Analysis
- (4) Intermediate Code generation
- (5) Code optimization.

(Compiler is basically used to convert the high level language to the low level language.)

Phases of Compiler Design



Lexical Analysis [Lexer, Tokenizer, Scanner]

- (1) Tokenization
- (2) Give Error messages
 - Exceeding length
 - unmatched string
 - illegal character.
- (3) Eliminate comments, white space (Tab, Blank space, Newline)

program:

```
int main() {
    1
    4
```

/* find Max of a and b */

```
int a = 20, b = 30;
```

```
if (a < b)
```

```
return (b);
```

```
else
```

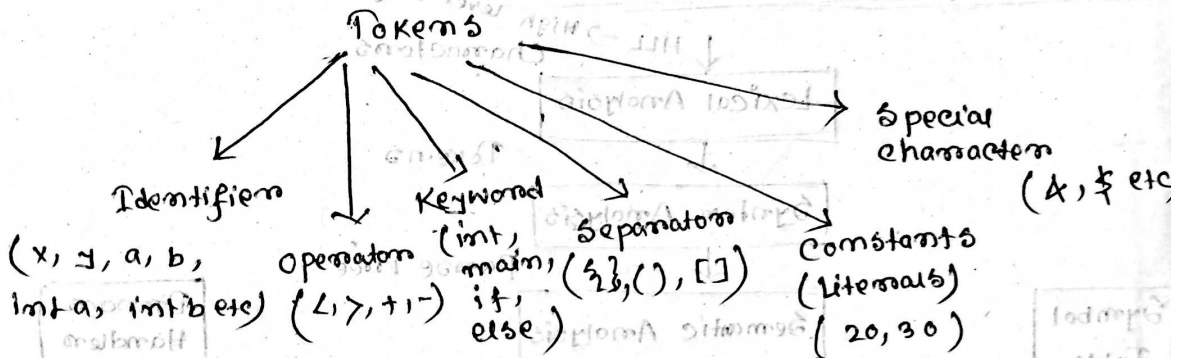
```
return (a);
```

```
}
```

Here in this program 32 tokens are present

32

We divide the whole program into tokens.



Q.1 `printf("i = %d, & i = %x", i, i);` → How many tokens are present?

⇒ 10

Q.2 `int main() {`

`x = y + z;`

`int x, y, z;`

`printf("Sum %d %d", x, y);`

⇒ ~~25~~ Tokens / 26 tokens

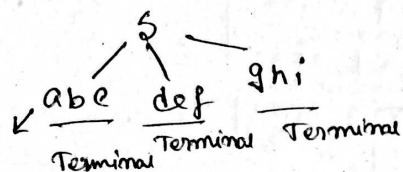
3. main() {
 1 2 3 4
 a = b + + + - - - + + + = = ;
 printf(" %d %d", a, b);
 16 17 18 19 20 21 22 23
 }
 24 25

Finding First() & Follow()

First(A) contains all terminals present in first place of every string derived by A.

- (1) $S \rightarrow abc | def | ghi$
- (2) $\text{First}(\text{terminal}) = \text{terminal}$
- (3) $\text{First}(\epsilon) = \epsilon$

$S \rightarrow abc | def | ghi \rightarrow$ In this grammar the, ~~First~~
 $\text{First}(S) = \{a, d, g\}$



$S \rightarrow \underline{abc} | \underline{ghi} | \underline{ijk}$ $\text{First}(S) = A$, Now $\text{First}(A) = \{a, b, c\}$
 $\therefore \text{First}(S) = \{a, b, c, g, i, j\}$

$A \rightarrow a | b | c$ $\text{First}(A) = \{a, b, c\}$

$B \rightarrow b$ $\text{First}(B) = \{b\}$

$D \rightarrow d$ $\text{First}(D) = \{d\}$

$S \rightarrow \cancel{abc} | \cancel{def} | \cancel{ghi}$

$\text{First}(S) = \{a, b, c, d, e, f, \epsilon\}$

$A \rightarrow a | b | \epsilon$

$\text{First}(A) = \{a, b, \epsilon\}$

$B \rightarrow c | d | \epsilon$

$\text{First}(B) = \{c, d, \epsilon\}$

$C \rightarrow e | f | \epsilon$

$\text{First}(C) = \{e, f, \epsilon\}$

$E \rightarrow TE'$

$\text{First}(E) = \text{First}(T) = \{id, c\}$

$E' \rightarrow *TE' | \epsilon$

$\text{First}(E') = \{*, \epsilon\}$

$T \rightarrow FT'$

$\text{First}(T) = \{id, c\}$

$T' \rightarrow \epsilon | +FT'$

$\text{First}(T') = \{\epsilon, +\}$

$F \rightarrow id | (E$

$\text{First}(F) = \{id, c\}$

Follow() :

Follow(A) Contains set of all terminals present immediate in right of A.

Rules : (1) Follow of start symbol is \$

$$Fo(A) = \{ \$ \}$$

$$(2) S \rightarrow ACD$$

$$C \rightarrow a|b$$

$$Fo(A) = \text{First}(C) = \{ a, b \}$$

$$Fo(D) = \text{Follow}(S) = \{ \$ \}$$

$$(3) S \rightarrow aSbS | bSaS | \epsilon$$

$$\text{Follow}(S) = \{ \$, b, a \}$$

* Follow never contains ϵ .

$$(4) S \rightarrow AaAb | BbBa$$

$$A \rightarrow \epsilon$$

$$Fo(A) = \{ a, b \}$$

$$B \rightarrow \epsilon$$

$$Fo(B) = \{ b, a \}$$

$$(5) S \rightarrow A \overset{\epsilon}{B} \overset{\epsilon}{C}$$

$$A \rightarrow DEF$$

$$B \rightarrow \epsilon$$

$$C \rightarrow \epsilon$$

$$D \rightarrow \epsilon$$

$$E \rightarrow \epsilon$$

$$F \rightarrow \epsilon$$

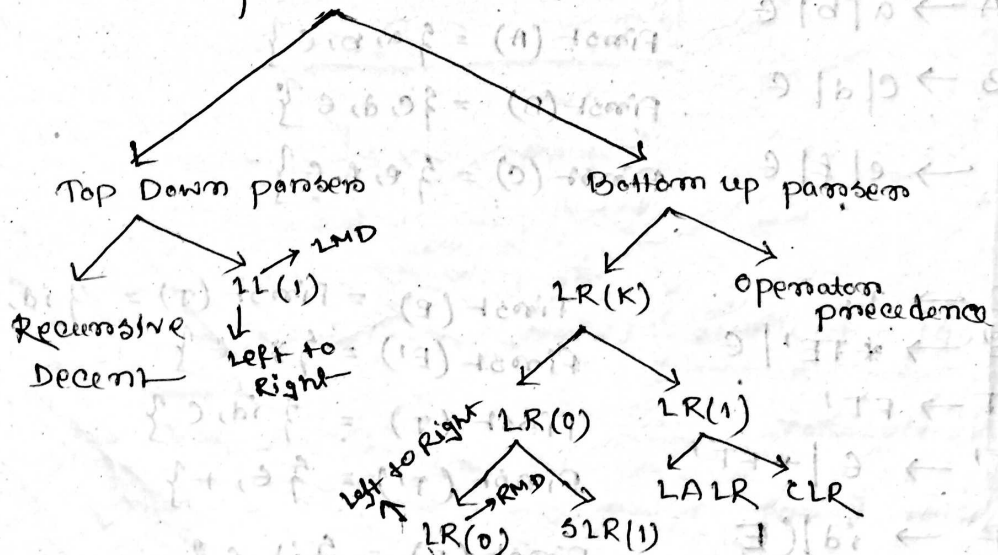
$$\text{Follow}(A) = \text{First}(B)$$

$$\text{First}(C)$$

$$\text{Follow}(\text{First}(S))$$

$$= \{ \$ \}$$

Parsers



Parsing :

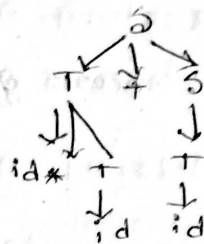
It is a process of deriving string from a given grammar.

Example:

$$S \rightarrow T + S \mid T$$

$$T \rightarrow id * T \mid id \mid (S)$$

$$w = id * id + id$$



$id * id + id$

LL(1) Grammar parsing Table:

$$\begin{array}{l} S \rightarrow (L) \mid a \\ L \rightarrow SL' \\ L' \rightarrow \epsilon \mid , SL \end{array} \quad \begin{array}{cc} \text{First} & \text{Follow} \\ \{ (, a \} & \{ \$,) \} \\ \{ (, a \} & \{) \} \\ \{ \epsilon, , \} & \{ , \} \end{array}$$

$$S \rightarrow aSbs \mid bSaS \mid \epsilon$$

	(	)	a	,	\$
S	1		2		
L	3		3		
L'		4		5	

LL(1)

	a	b	\$
S	1/3	2/3	3

Not LL(1)

using parsing table we can say a table is LL(1) or not.

Check whether the grammar is LL(1) or not?

① $aSa \mid bS \mid c$

If there is no null production, then,

$$S \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3 \quad \text{No null } \epsilon$$

$$F(\alpha_1) \cap F(\alpha_2) \cap F(\alpha_3) = \emptyset \quad \text{LL(1)}$$

$$\neq \emptyset \quad \text{NOT LL(1)}$$

If there is a null production, then,

$$S \rightarrow \alpha_1 \mid \alpha_2 \mid \epsilon$$

$$F(\alpha_1) \cap F(\alpha_2) \cap \text{Follow}(S) \text{ if it is equal to } \emptyset$$

then it is LL(1)

If it's not equal to $\emptyset$ then it is not LL(1)

Now, $S \rightarrow (a \delta a) | b \delta | c$
 $\alpha_1 \quad \alpha_2 \quad \alpha_3$

$a \quad \cap \quad b \quad \cap \quad c$

There are no common part means $\emptyset = \emptyset$

$\therefore$ It is a LL(1) grammar.

$\delta \rightarrow iC \pm \delta \delta_1 | a$ $i \cap a = \emptyset$

$\delta_1 \rightarrow e \delta | \epsilon$

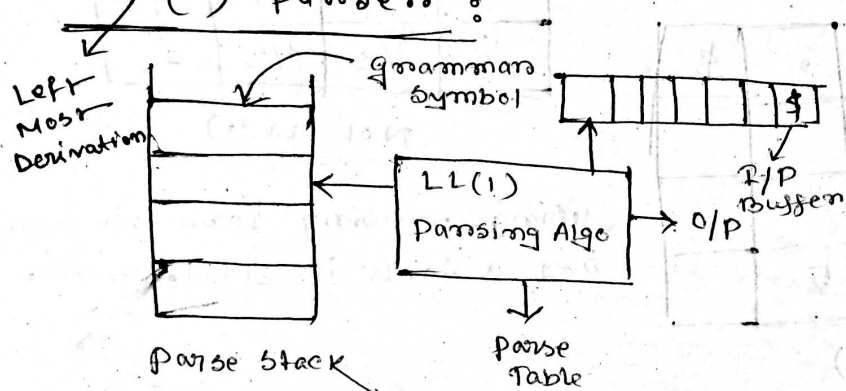
$c \rightarrow b$

$e \cap e \neq \emptyset$

$\therefore$ It is not a LL(1) grammar.

→ left to right

LL(1) parser:

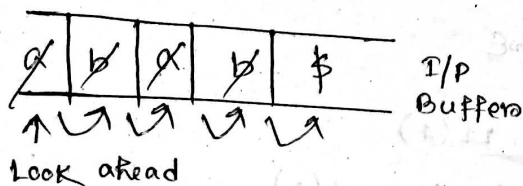


Example:

$S \rightarrow AA$ 1

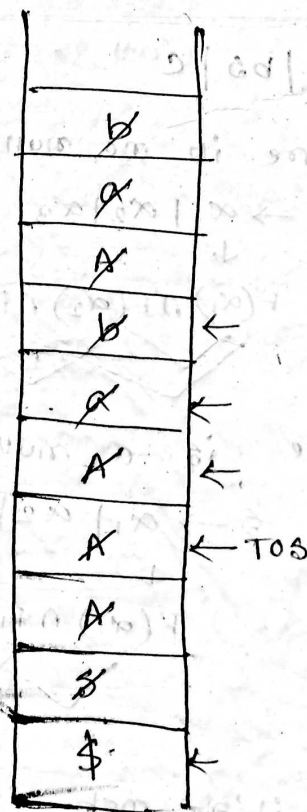
$A \rightarrow aA$ 2

$A \rightarrow b$ 3



	a	b	\$
S	$S \rightarrow AA$ 1	$S \rightarrow AA$ 1	
A	$A \rightarrow aA$ 2	$A \rightarrow b$ 3	

Parse Table.



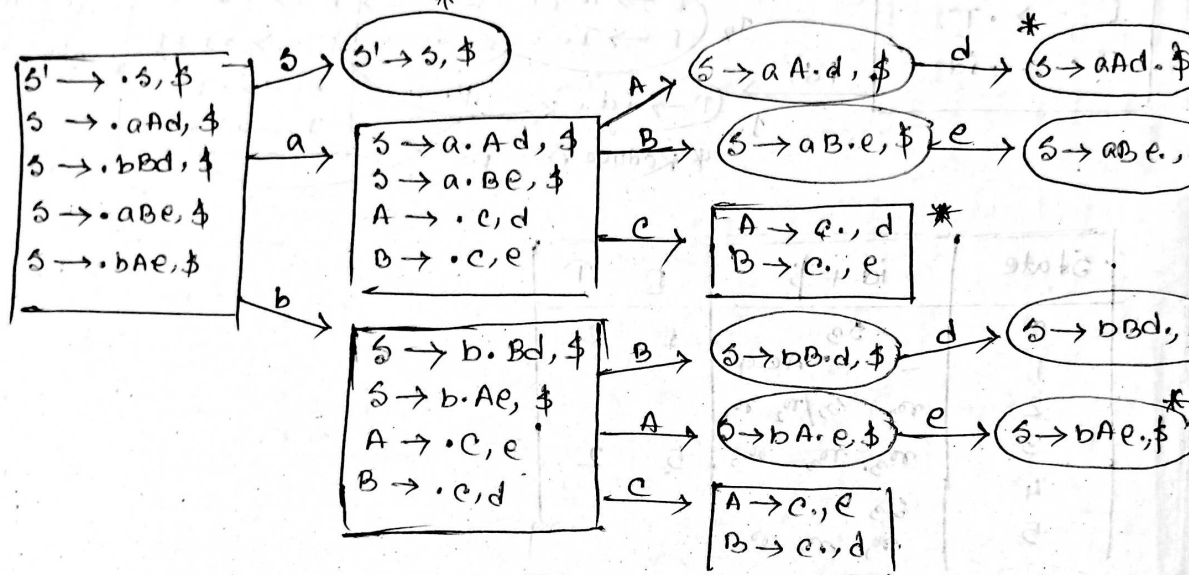
$\therefore$ This string is accepted string.

CLR Parsing Table (LR1 canonical items)

$S \rightarrow aAd \mid bBd \mid aBe \mid bAe$
 $A \rightarrow c$
 $B \rightarrow c$

LALR —
 CLR — LR(1)

$S' \rightarrow \cdot S, \$ \rightarrow \text{Look Ahead}$
 $S \rightarrow \cdot aAd, \$$
 $S \rightarrow \cdot bBd, \$$
 $S \rightarrow \cdot aBe, \$$
 $S \rightarrow \cdot bAe, \$$

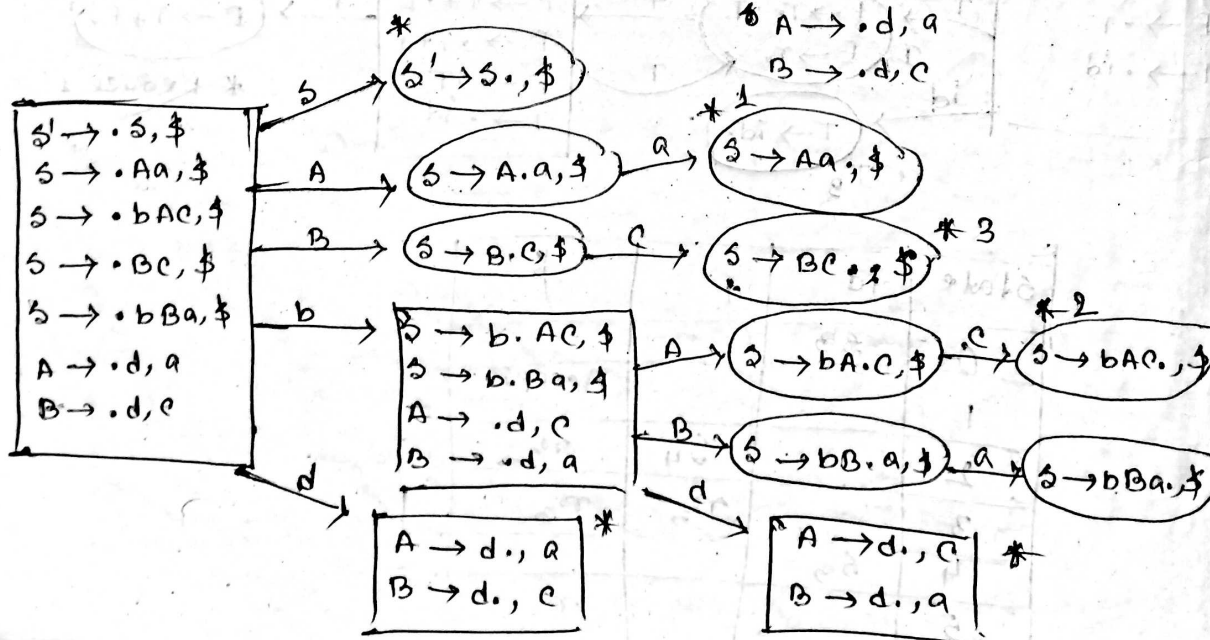


LALR parsing Table: (Look Ahead LR)

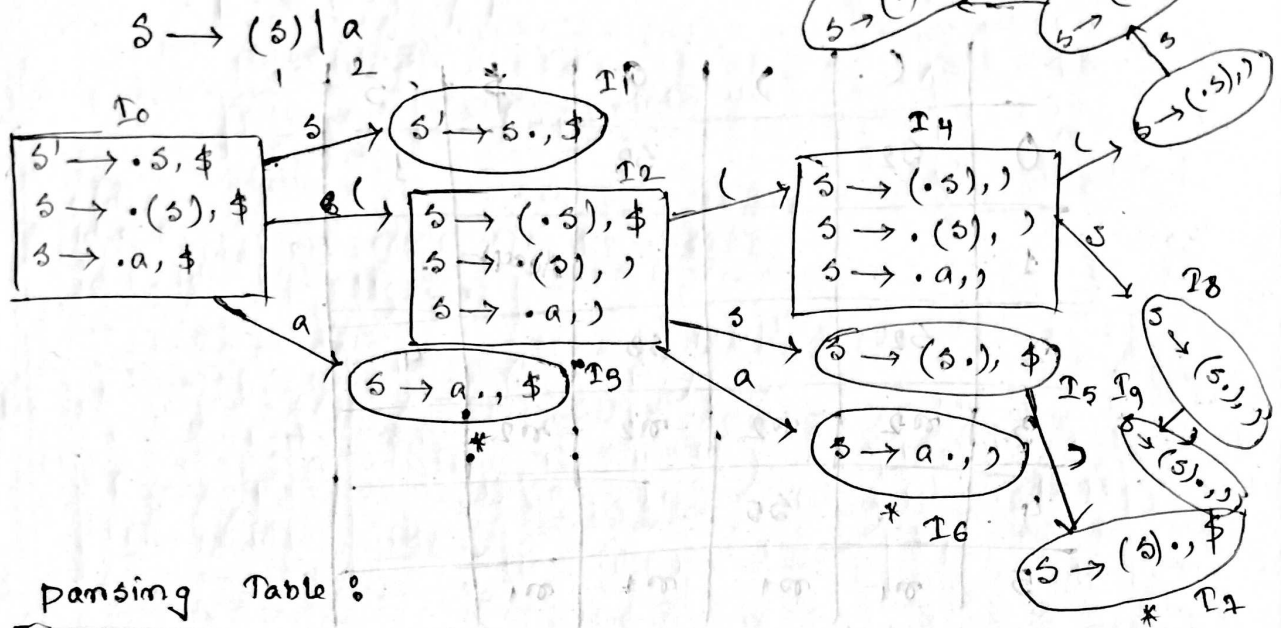
$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$
 $A \rightarrow d \ 5$
 $B \rightarrow d \ 6$

$S' \rightarrow \cdot S, \$$
 $S \rightarrow \cdot Aa, \$$
 $S \rightarrow \cdot bAc, \$$
 $S \rightarrow \cdot Bc, \$$
 $S \rightarrow \cdot bBa, \$$
 $A \rightarrow \cdot d, a$
 $B \rightarrow \cdot d, c$

Extended version of CLR



Difference between CLR and LALR:



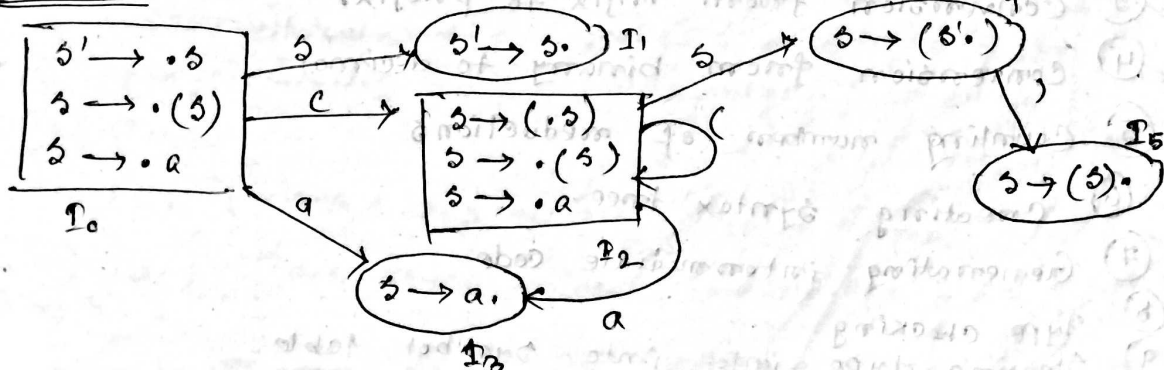
Transition Table:

	(	)	a	\$	s
0	2		3		1
1			Accept		
2	4		6		5
3			2		
4	4		6		8
5			7		
6			2		
7			1	1	
8			9		
9			1		

LR(0)

$S \rightarrow (s) | a$

Accept



parsing Table :

	(	)	a	\$	5
0	s2		s3		1
1				Accept	
2	s2		s3		4
3	m2	m2	m2	m2	
4		s5			
5	m1	m1	m1	m1	

SLR parsing Table of the same example :

	(	)	a	\$	5
0	s2		s3		1
1				Accept	
2	s2		s3		
3		m2		m2	
4		s5			
5		m1		m1	

Semantic Analysis

↓ PT
[SA]

↓
Annotated
parse tree

SDT (Syntax Directed Translation) its applications

Find Logic
4 Actur
mean
of a
gram

Applications :

Grammar + Semantic. = SDT Rules

- ① Executing Arithmetic Expression.
- ② Conversion from infix to postfix.
- ③ Conversion from infix to prefix.
- ④ Conversion from binary to decimal
- ⑤ Counting number of reductions
- ⑥ Creating syntax tree.
- ⑦ Generating intermediate code
- ⑧ Type checking
- ⑨ Storing type info into symbol table.

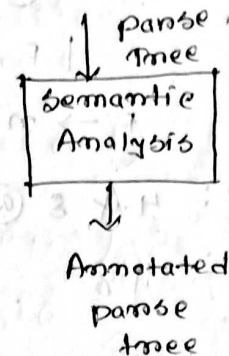
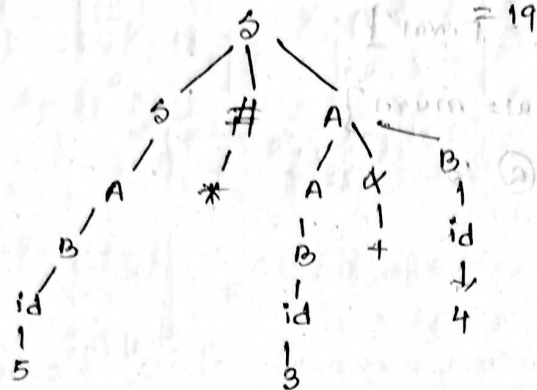
Grammar Rules

$$S \rightarrow S \# A / A \quad \{ S.val = S.val * A.val ; \}$$

$$A \rightarrow A \& B / B \quad \{ A.val = A.val + B.val ; \}$$

$$B \rightarrow id \quad \{ B.val = id.val ; \}$$

Given string $5 \# 3 \& 4 = 5 * 3 + 4$



Types of evaluating SDT (Syntax Directed Translation)

- (i) Top down
- (ii) Bottom up

Given grammar:

$S \rightarrow AS \{ print f(1) \}$ — ①

$S \rightarrow AB \{ print f(2) \}$ — ②

$A \rightarrow a \{ pf(3) \}$ — ③

$B \rightarrow bc \{ pf(4) \}$ — ④

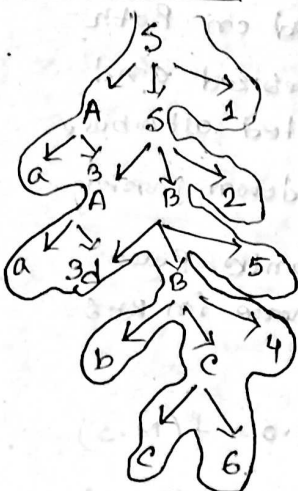
$B \rightarrow dB \{ pf(5) \}$ — ⑤

$C \rightarrow c \{ pf(6) \}$ — ⑥

$I/P = aadbc$. Then what is the o/p.

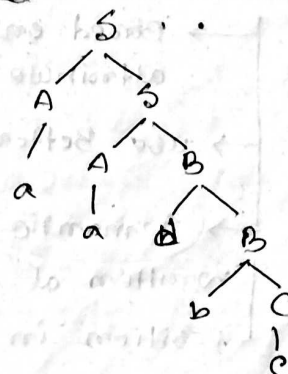
First make the parse tree.

Top down



O/P = 3 3 6 4 5 2 1

Bottom up



O/P = 3 3 6 4 5 2 1

How to Evaluate Arithmetic Expression using SDT.

$$E \rightarrow E \wedge T \quad \{ E.val = E.val * T.val \}$$

$$| T \quad \{ E.val = T.val \}$$

$$T \rightarrow T @ F \quad \{ T.val = T.val - F.val \}$$

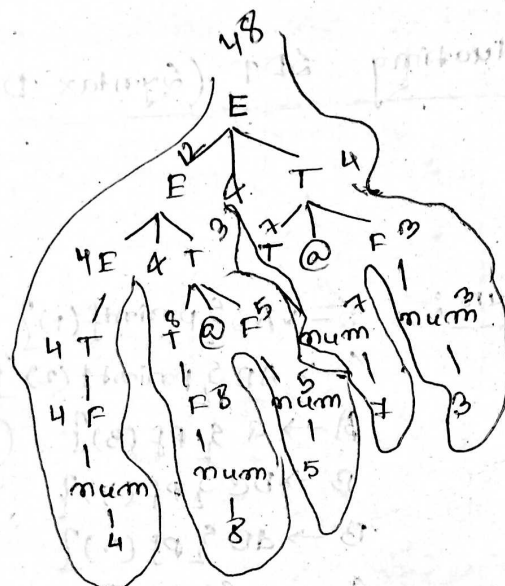
$$| F \quad \{ T.val = F.val \}$$

$$F \rightarrow num \quad \{ F.val = num \}$$

I/P: 4 4 8 @ 5 4 * @ 3

O/P: 48

using Bottom up:



Types of SDT

S-Attributed SDT

- Based on synthesized attribute
- use Bottom up parsing
- Semantic Rules always written at Rightmost position in R.H.S.

L-Attributed SDT

- Based on Both synthesized and inherited attribute
- Top down parsing
- Semantic Rules anywhere in R.H.S.

$$A \rightarrow LM \quad \{ L.i = l(A.i); M.i = m(L.s); A.s = f(M.s) \}$$

$$A \rightarrow QR \quad \{ R.i = r(A.i); Q.i = q(R.s); A.s = f(Q.s) \}$$

Q: $A \rightarrow PQ$ and $A \rightarrow XY$

Rule 1: $P.i = A.i + 2$, $Q.i = P.i + A.i$ and $A.s = P.s + Q.s$

Rule 2: $X.i = A.i + Y.s$ and $Y.i = X.s + A.i$

which of the following is true?

- (A) Both Rules are L-Attributed
- (B) Only R1 is L-Attributed
- (C) Only R2 is L-Attributed
- (D) Neither R1 Nor R2 L-Attributed

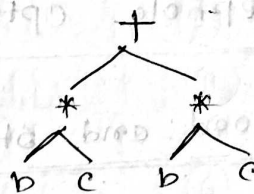
⇒ Only R1 is L-Attributed.

Intermediate Code Generation:

- Machine Independent
- Abstract Syntax Tree
- Direct Acyclic Graph
- postfix
- 3-Address code **

Example: $(b * c) + (b * c)$

Abstract Syntax Tree:



Direct Acyclic Graph:



Postfix: $bc * bc * +$

3-Address code:

$t_1 = b * c$
 $t_2 = b * c$
 $t_3 = t_1 + t_2$

Maximum 3 Address.

3- Address code : $(x = y \text{ op } z)$
 $\rightarrow$ opcode

$$x = (a * b) + (c * d)$$

Various statements in 3 address code

① Assignment :

- $\rightarrow x = y \text{ op } z$
- $\rightarrow x = \text{op } y$
- $\rightarrow x = y$

$\rightarrow$ Relational operator

② Jump

- $\rightarrow$ Conditional if $x \text{ rel op } y \text{ goto } L$
- $\rightarrow$ unconditional goto L

③ Array Assignment $\rightarrow x = y[i]$
 $\rightarrow x[i] = y$

it provide flexibility

④ Pointer, address assign $\rightarrow x = \&y$
 $\rightarrow x = *y$

$$\begin{aligned} t_1 &= a * b \\ t_2 &= c * d \\ x &= t_1 + t_2 \end{aligned}$$

Peephole = small window
 • small piece of code
 • Machine Independent

Peephole optimization

① Redundant load and store :

~~a~~ $a = b + c$
 $d = a + e$

```

Mov  b, R0
Add  c, R0
Mov  R0, a
Mov  a, R0
Add  e, R0
Mov  R0, d
    
```

② Strength Reduction :

$$x^2 \Rightarrow x * x$$

$$x * 2 \Rightarrow \text{left shift}$$

$$x / 2 \Rightarrow \text{Right shift}$$

③ Simplify Algebraic Expressions :

$a := a + 0$

$a := a * 1$

$a := a / 1$

$a := a - 0$

④ Replace slower Instructions with faster :

Add #1, R $\Rightarrow$ INC R

Sub #1, R $\Rightarrow$ DEC R

$\Rightarrow$ $\left. \begin{array}{l} a \text{ Load } x \\ a \text{ Load } x \\ \text{Mul} \end{array} \right\} \Rightarrow \begin{array}{l} a \text{ Load } x \\ \text{Dup} \\ \text{Mul} \end{array}$

⑤ Deadcode Elimination :

```
int dead(void)
```

```
{  
  int a = 10;
```

```
  int b = 20;
```

```
  int c;
```

```
  c = a * 10;
```

```
  return c;
```

```
  b = 30;
```

```
  b = b * 10;
```

```
  return 0;
```

```
}
```

} deadcode

Code optimization

Platform Dependent Techniques

- Peephole optimization
- Instruction level parallelism
- Data level parallelism
- Cache optimization
- Redundant Resources

Platform Independent Techniques

- Loop optimization
 - Loop unrolling
- Code Movement, frequency reduction, loop jamming
- Constant folding
- Constant propagation
- Common subexpression Elimination

Loop optimization :

(1) Code Motion (Frequency Reduction)

```
a=100;
while (a>0)
{
    x=a+k;
    if (a/x == 0)
        printf ("%d", a);
}
```

(2) Loop fusion

```
int i, a[100], b[100]
for (i=0; i<100; i++)
    a[i]=1;
for (i=0; i<100; i++)
    b[i]=2;
```

(3) Loop unrolling

```
for (i=0; i<5; i++)
    printf ("varun");
```

Question on code optimization :

```
for (i=0; i<n; i++)
{
    for (j=0; j<n; j++)
    {
        if (i%2)
        {
            x += ((4*j) + 5*i);
            y += ((4*j) + 5*i);
        }
    }
}
```

True

(High frequency code $\rightarrow$ Low frequency code)
invariant computation

(b) Contain common subexpression elimination True

(c) Scope of strength reduction True

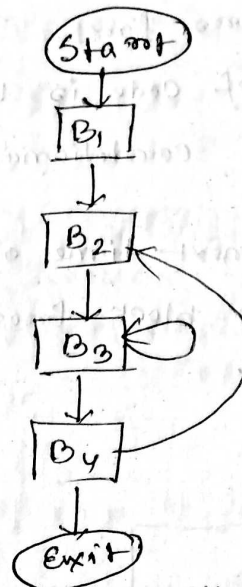
(d) Scope of deadcode elimination

Which of the above statement is false?

What is Control Flow Graph:

Consider the intermediate code:

1. $i = 1$ Block 1
2. $j = 1$ Block 2
3. $t_1 = 5 * i$
4. $t_2 = t_1 + j$
5. $t_3 = 4 * t_2$
6. $t_4 = t_3$
7. $a[t_4] = -1$
8. $j = j + 1$
9. if $j \leq 5$ goto (3)
10. $i = i + 1$
11. if $i \leq 5$ goto (2)



4 block and 5 edges.

6 nodes and 7 edges.

Find the no. of nodes and edges in control flow graph?

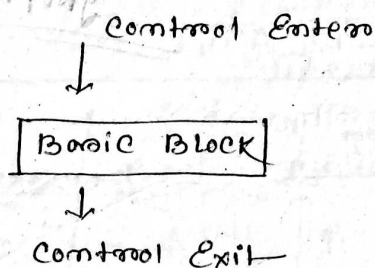
How to find leaders in basic block?

Step 1: First statement is always a leader.

Step 2: Address of conditional, unconditional goto are leaders.

Step 3: Next line of conditional and unconditional goto are leaders.

Basic block properties:



* First line of basic block is leader.

What is Basic block and how to partition a code into basic block.

→ Sequence of intermediate code with single entry and single exit.

→ No. jumps in the middle.

Algorithm to make basic block:

- ① Identify leaders, first.
- ② First line of code is leader.
- ③ Address of conditional, unconditional goto are leaders.
- ④ Immediate next line of goto are leaders.
- ⑤ Make basic block from leader to line before next leader.

✓(1) $i = 1$ B1

✓(2) $j = 1$ B2

✓(3) $t_1 = 10 \times j$

(4) $t_2 = t_1 + j$

(5) $t_3 = 8 \times t_2$

(6) $t_4 = t_3 - 88$

(7) $a[t_4] = 0.0$

(8) $j = j + 1$

(9) if $j \leq 10$ goto (3)

✓(10) $i = i + 1$

(11) if $i \leq 10$ goto (2)

✓(12) $i = 1$ B5

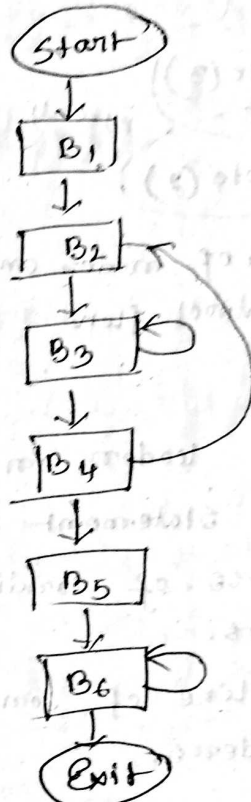
✓(13) $t_5 = i - 1$

(14) $t_6 = 88 \times t_5$

(15) $a[t_6] = 1.0$

(16) $i = i + 1$

(17) if $i \leq 10$ goto (13)



Control flow graph

B3

B4

B6

Slope of optimization

Local optimization

- within Basic Block
- Local Common Subexpression elimination

$$\begin{aligned} x &= y * z + k \\ A &= y * z - M \end{aligned} \rightarrow \begin{aligned} t_1 &= y * z \\ x &= t_1 + k \\ A &= t_1 - M \end{aligned}$$

- Deadcode Elimination within BB

```
if (0)
{
    printf("Dead");
}
```

- Constant propagation

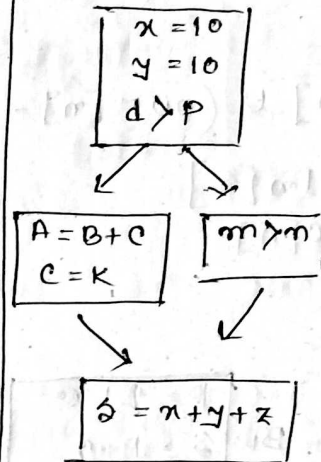
$$\begin{aligned} x &= 10 \\ A &= B + C \\ z &= x + y \end{aligned} \rightarrow \begin{aligned} A &= B + C \\ z &= 10 + y \end{aligned}$$

- Algebraic law (commutative, Associative)

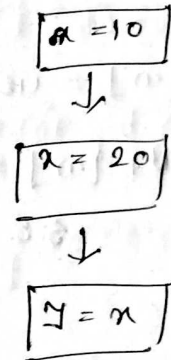
- Reordering statements

Global optimization

- Between basic blocks
- Extended to entire control flow graph.



Constant propagation b/w basic blocks



Deadcode Elimination

Liveness Analysis

whether a variable is live or not?

node	use	DEF	IN	OUT	IN - Def	OUT
1	q, m, v	p, m, u	q, m, v	m, u, s	m, q, v	m, u, s, v
2	m, u	v	m, u	m, u, m	m, u	m, v
3	s, u	q	s, u	v, s, u	v, m, s, u	m, v
4	v, m	q	v, m	q, m, v	m, v	m, q, v

$IN[B]$ = set of variable live at beginning of B

$OUT[B]$ = set of variable n " Just after B

$USE/kill[B]$ = variables that are used in B before any assignment.

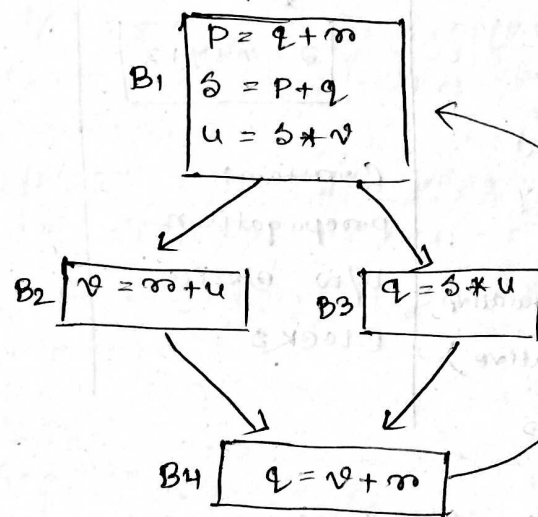
(variables in R.H.S but not in L.H.S of previous statement in B)

$DEF/KILL[B]$ = variables that are assigned a value in B (variables in L.H.S)

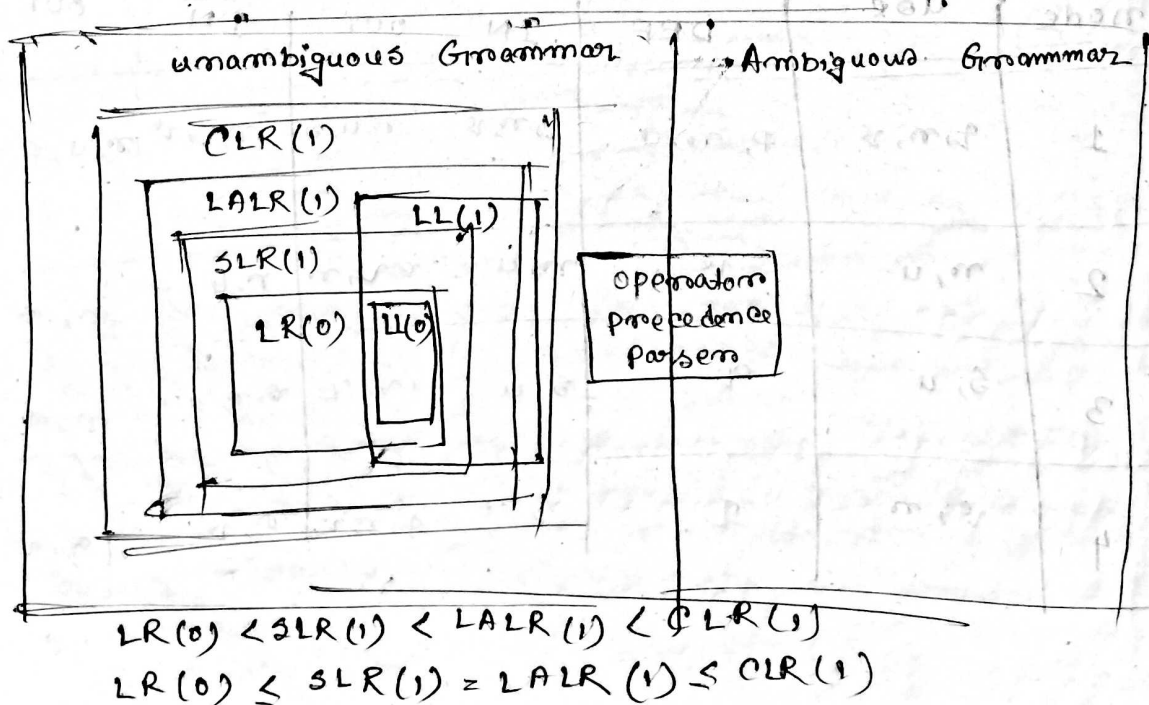
$$IN[n] = use[n] \cup (out[n] - DEF[n])$$

$$OUT[n] = \bigcup_{s \in succ[n]} IN[s]$$

$$s \in succ[n]$$



Comparison of all parsers:



Symbol Table

It is a data structure.

Name	Type	Size	Dimension	Line of Declaration	Line of usage	Address
x	int	4	0	4	6-10	-
a	Char	5	1	3	-	-

Dimension mainly use when we have array. 'x' is one dimensional array so dimension of x is 1.

L1 main()

L2 {

L3 char a[5];

L4 int x;

L5 }
L6 x

L7 ;

L8 x

L9 }

L10 }

Errors in Lexical Analysis & their Recovery :

Types of Errors detected by Lexical Analyzer

1. Long Identifiers.
2. Too long Numerical literals.
3. Badly formed numerical literals.
4. Input characters that are not present in source language.
5. Spelling Mistakes.

Symbol Table Implementation :

Access time of the symbol table will be logarithmic, if it is implemented by a —

1. Linear list
2. Search tree
3. Hash table
4. none of the above.

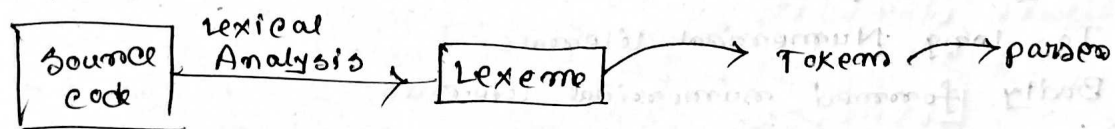
Data Structure	Intention	Lookup Time
Unordered Array	$O(1)$	$O(n)$
Ordered Array	$O(n)$	$O(\log n)$
Unordered Linked List	$O(1)$	$O(n)$
Ordered Linked List	$O(n)$	$O(n)$
Search Tree	$O(\log n)$	$O(\log n)$
Hash Table	$O(1)$	$O(1)$

Lexeme vs Tokens

Lexeme: A lexeme is a sequence of characters from the input that match a pattern.

Tokens: Tokens are symbolic names for the entities that make up the text of the program.

Ex: ID, Constant, Keywords, operators, punctuation, Literal, String.



Ex:	Lexeme	Tokens
int x = 5;	int	Keyword
	x	identifier
	=	operator
	5	constant
	;	operator

Derivation Tree / parse Tree in TOC 4

Compiler:

- It is basically the sequence of steps followed to generate a word (string) from a given Grammar.

$$G: E \rightarrow E + E \mid E * E \mid E = E \mid id$$

$$w: id + id * id$$

$\{ \lambda v, T, P, \delta \}$
 Whether this word belongs to this grammar or not?

⇒

$$E \rightarrow E + E$$

↓

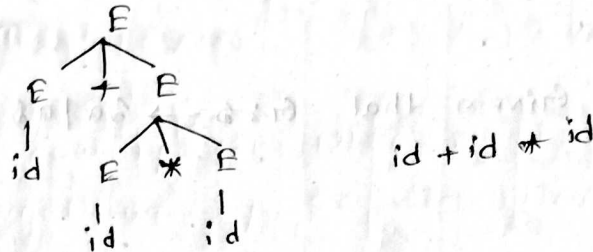
$$E + E * E$$

id + id * id → Yes, this string is

basically belongs to this grammar.

→ This sequence of steps is called derivation.

parse Tree



Recursive and Non-Recursive Context-free Grammar

Recursive CFG: It generates Infinite no. of strings.

for example:

$$(1) \begin{aligned} S &\rightarrow Sa \\ S &\rightarrow b \end{aligned} \quad \{ b, ba, bba, bbba, \dots \infty \}$$

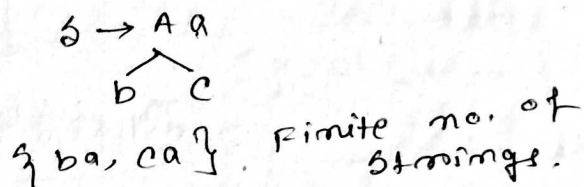
$$(2) \begin{aligned} S &\rightarrow aS \\ S &\rightarrow b \end{aligned} \quad \{ a, ab, aab, aaab, \dots \infty \}$$

$$(3) \begin{aligned} S &\rightarrow Aa \\ A &\rightarrow Ab | c \end{aligned} \quad \{ ca, cba, ccba, \dots \infty \}$$

Non-Recursive CFG: It generates finite no. of strings.

for example:

$$\begin{aligned} S &\rightarrow Aa \\ A &\rightarrow b | c \end{aligned}$$



Ambiguous & unambiguous Grammar:

- For ambiguous there exists more than one derivation for any word that belongs to Grammar.
- For unambiguous there exist exactly one derivation for any word that belongs to Grammar.

For example: Given that $G: S \rightarrow Sa | aS | a$

Interme
Gram

- ① Sym
- ② DA
- ③ 3-A

$P =$

$t_1 =$

$t_2 =$

$t_3 =$

$t_4 =$

$t_5 =$

$t_6 =$

Quad

① Syntax Directed Tree

② DAG

③ 3-Address Code

Quadruples

Triples

Indirect Triples

$$P = (q+r) * (q-r) + (q-r) * s + s/u$$

$$t_1 = q+r$$

$$t_4 = s/u$$

$$t_2 = q-r$$

$$t_8 = t_6 + t_7$$

$$t_3 = t_1 * t_2$$

$$P = t_8$$

$$t_4 = q-r$$

$$t_5 = t_4 * s$$

$$t_6 = t_3 + t_5$$

Quadruples

Operation	Operand 1	Operand 2	Result
Addition	q	r	t ₁
Subtraction	q	r	t ₂
Multiplication	t ₁	t ₂	t ₃
Subtraction	q	r	t ₄
Multiplication	t ₄	s	t ₅
Addition	t ₃	t ₅	t ₆
Division	s	u	t ₇
Addition	t ₆	t ₇	t ₈
Assignment	t ₈	-	P

Triples

Address	Operation	Operand 1	Operand 2 / Result
(1)	Addition	q	r
(2)	Subtraction	q	r
(3)	Multiplication	(1)	(2)
(4)			

EX-2

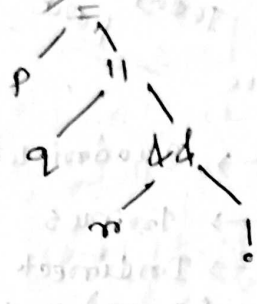
$P = 211 \text{ m } 44 (15)$

$t_1 = 15$ 3-Address code

$t_2 = \text{m } 44 \text{ t}_1$

$t_3 = 211 \text{ t}_2$

$t_4 = \boxed{P = t_3}$



Quadruple

operation	op 1	op 2	Result
logical not	5	—	t_1
logical and	m	t_1	t_2
logical or	q	t_2	t_3
Assignment	P	—	t_3

Triples

Address	operation	op 1	Result/op 2
(1)	logical not	5	t_1
(2)	logical and	m	t_1
(3)	OR	q	(2)
(4)	Assignment	(3)	P

EX-3

if $(P > 5)$

$q = q + 1;$

else

$m = m - 1;$

$\Rightarrow t_1 = P > 5$

if t_1 goto t_1

goto t_2

L1: $t_2 = q + 1$

$q = t_2$

goto t_3

L2:

$t_3 = m - 1$

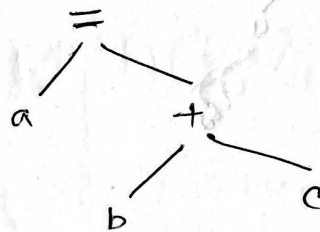
$m = t_3$

goto t_3

L3: ...

	operation	OP 1	OP 2	Result
		P	5	t ₁
	Boolean	t ₁	-	goto t ₁
	-	-	-	goto t ₂
21:	Add	q	1	t ₂
	Assign	t ₂	-	q
	-	-	-	goto 23
22:				
23:				

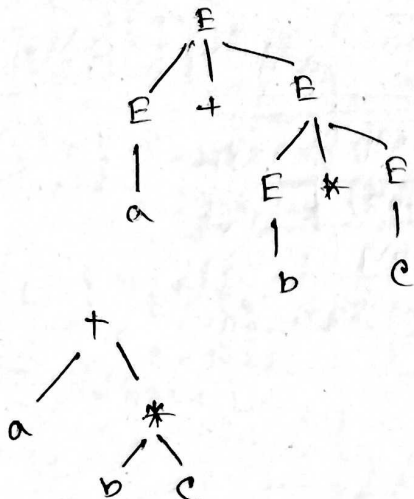
Today's First part



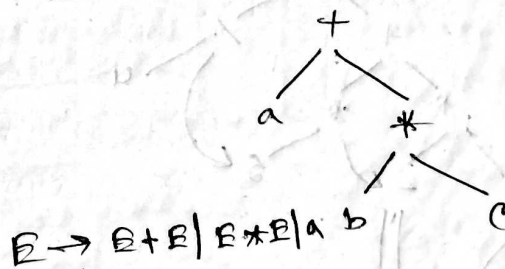
- ① Syntax Directed Tree/Graph
- ② DAG
- ③ 3-Address Code
 - Quadruples (4 columns)
 - Triples
 - Indirect Triples

Annotated parse Tree

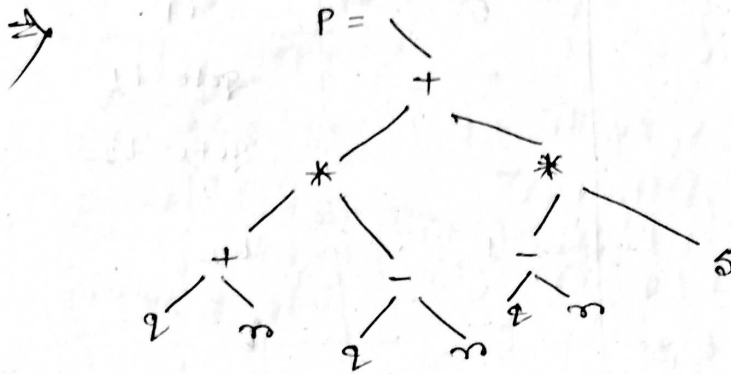
$a + b * c$



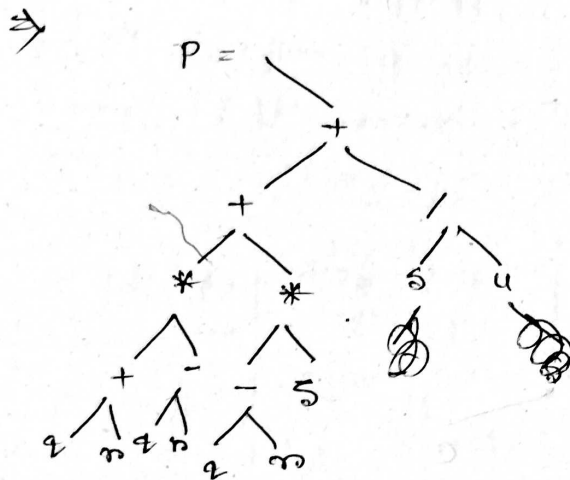
Syntax Directed Tree



$$P = (2+m) * (2-m) + (2-m) * 5$$



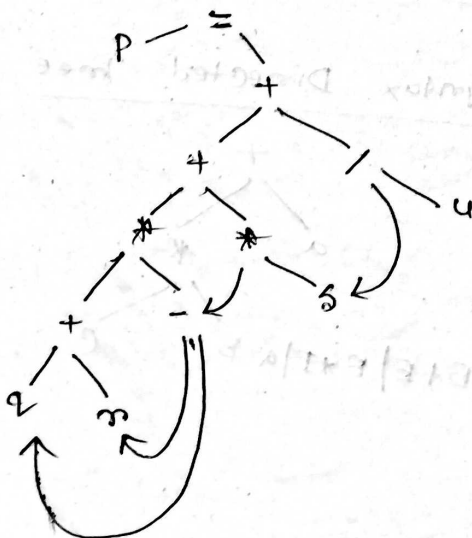
$$P = (2+m) * (2-m) + (2-m) * 5 + 5/u$$



$$5/u + (2+m) * (2-m) + (2-m) * 5$$

→ Not correct if addition is
L to R associativity.

DAG:



while ($P > 5$)

```
{
  q = q + 1;
  m = m - 1;
}
```

$t_1 = P > 5$

L0: if t_1 goto L1
 " " goto L2

L1: $t_2 = q + 1$
 $q = t_2$
 $t_3 = m - 1$
 $m = t_3$
 goto L0

L2:

28/04/23

(1) if-else

(2) While

(3) for ($i=0; i < m; i++$)

```
{
  if ( $a > 5 \wedge 4 < b$ )
```

```
{
  p = p + 1;
}
```

}

$i = 0$

L0: if $i < m$ goto L1
 goto L2

L1: if $a > 5$ goto L3
 goto L4

L3: if $b < 7$ goto L5
 goto L6

L5: $t = p + 1;$
 $p = t$
 goto L0

(2) while ($x > 5$)

```
{
  p = p + 1;
}
```

L0: if $x > 5$ goto L1

goto L2

L1: $t = p + 1;$
 $p = t$
 goto L0

L2:

③ Switch (a):

Case 1: $P = P + 1$;

break;

Case 2: $Q = Q + 1$;

break;

Case 3: $X = X + 1$

Case 4: $Y = Y + 1$;

}

⇒ if $a == 1$ goto L1
~~goto L2~~

if $a == 2$ goto L2

if $a == 3$ goto L3

if $a == 4$ goto L4

goto L5

L1: $t = P + 1$

L2: ~~L5~~

$P = t$

goto L5

L3: $t3 = X + 1$

$X = t3$

goto L4

L4: $Y + 1$;

L5:

$Y = t4$

goto L5

④ if ($a > 5$ & $b < 6$)

$P = P + 1$;

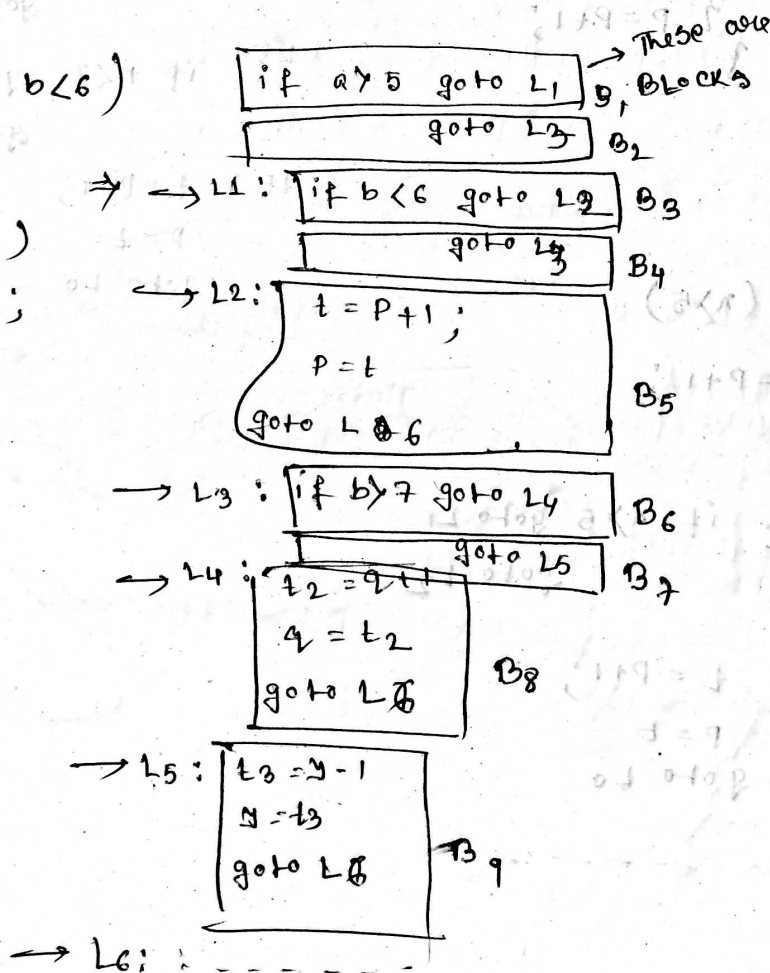
else if ($b > 7$)

$Q = Q + 1$;

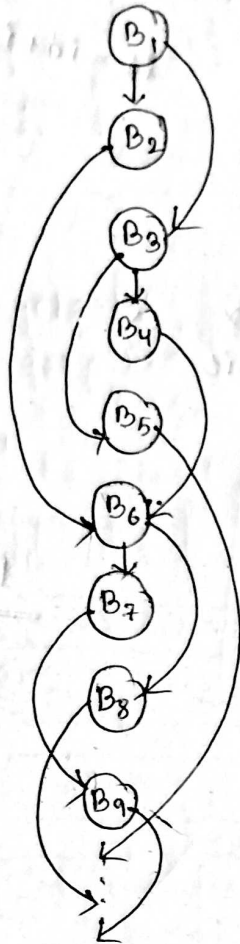
else

$Y --$;

L1, L2, L3, L4, L5
are leaders



Dependency graph :



Lexical Analyzer :

$x = a + b * c$

↓
Lexical Analysis

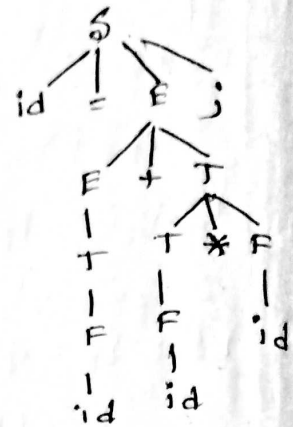
Lexemes	Tokens
x	identifier
=	operator
a	identifier
+	operator
b	identifier
*	operator
c	identifier

Syntax Analyzer;

Lexemes	Tokens
x	identifier
=	operator
a	identifier
+	operator
b	identifier
*	operator
c	identifier

⇒ Syntax Analysis ⇒

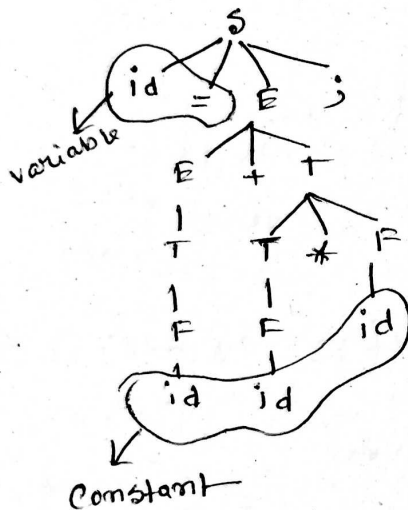
id = id + id * id ;



Parse Tree

$x = a + b * c$
 $\Rightarrow S \rightarrow id = E ;$
 $E \rightarrow E + T | T$
 $T \rightarrow T * F | F$
 $F \rightarrow id$

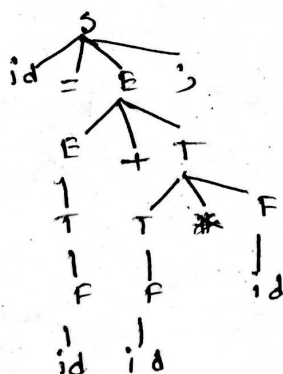
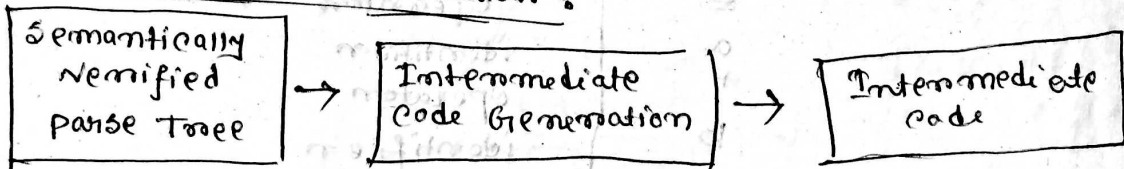
Semantic Analyzer :



⇒ Semantic Analysis ⇒

Semantically Verified Parse Tree.

Intermediate Code Generation :



id = id + id * id

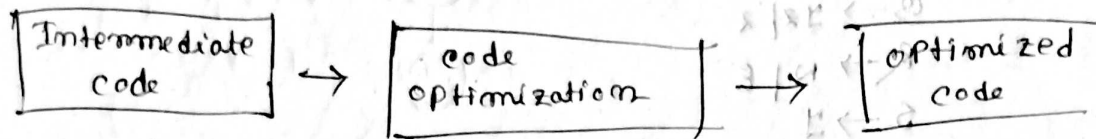
↓↑

x = a + b * c

Three Address code (TAC)

1. $t_0 = b * c ;$
2. $t_1 = a + t_0 ;$
3. $x = t_1$

Code optimizer:



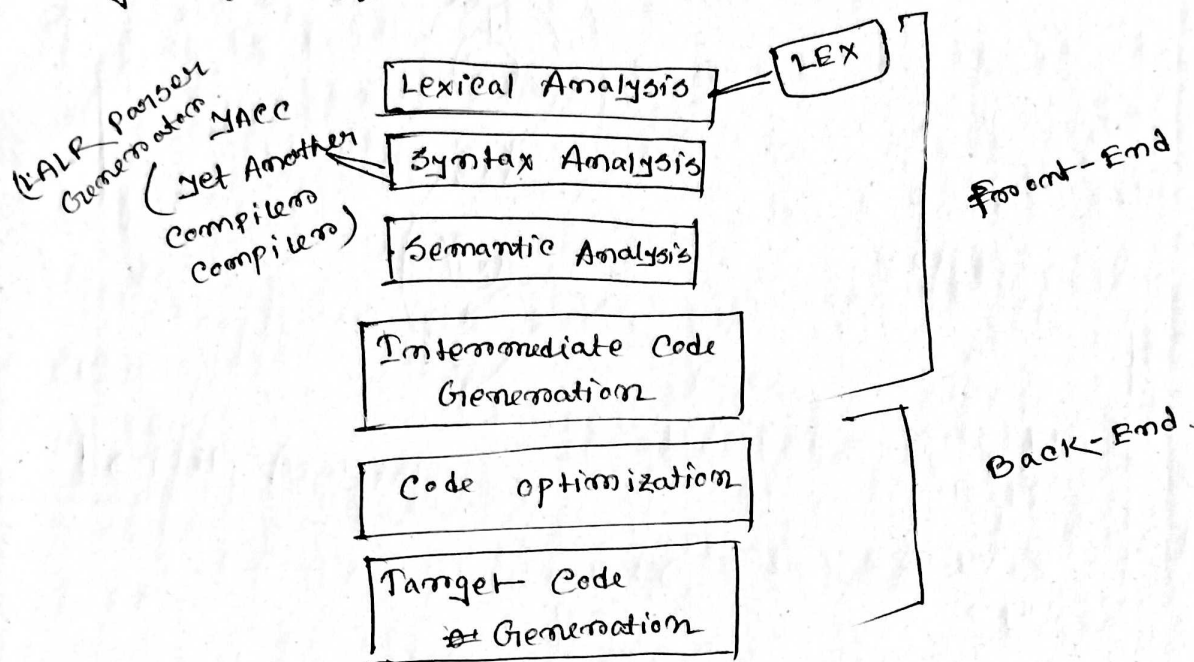
Three Address code (TAC)

1. $t_0 = b * c;$
2. $t_1 = a + t_0;$
3. $x = t_1;$

Three Address code (TAC)

1. $t_0 = b * c;$
2. $x = a + t_0;$

• Target code Generation



Derivation of FIRST :

$E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' \mid \epsilon$
 $F \rightarrow id \mid (E)$

$\Rightarrow$
 $FIRST(F) = \{id, (\}$
 $FIRST(T') = \{*, \epsilon\}$
 $FIRST(T) = \{id, (\}$
 $FIRST(E') = \{+, \epsilon\}$
 $FIRST(E) = \{id, (\}$

FOLLOW:

	FOLLOW(E)	FIRST	FOLLOW
$E \rightarrow TE'$		$\{id, (\}$	$\{ \$,) \}$
$E' \rightarrow +TE' \mid \epsilon$		$\{ +, \epsilon \}$	$\{ \$,) \}$
$T \rightarrow FT'$		$\{id, (\}$	$\{ +, \$,) \}$
$T' \rightarrow *FT' \mid \epsilon$		$\{ *, \epsilon \}$	$\{ +, \$,) \}$
$F \rightarrow id \mid (E)$		$\{id, (\}$	$\{ *, +, \$,) \}$

1. Consider the following grammar:

$$P \rightarrow xQRs$$

$$Q \rightarrow yz | z$$

$$R \rightarrow w | \epsilon$$

$$S \rightarrow \lambda$$

$$\Rightarrow \text{Follow}(Q) = \text{First}(R) \\ = \{w, \epsilon\} \cup \text{First}(S)$$

